
Exit Codes Are Not Booleans:

On the Misuse of `if` in POSIX Shell Scripting

Ivan Gaydardzhiev

Independent Research 2026

Abstract

The `if` construct in POSIX shell is not a boolean evaluator. It is a *command runner* that branches on exit status. When programmers write `if [ condition ]`, they invoke a separate utility, the `[` command, to encode a boolean result as an exit code, which `if` then decodes. This is a category error: value-based dispatch is being performed through a process boundary. The `case` construct, by contrast, dispatches on a value directly, with no external command, no process overhead, and no encoding round-trip. This paper argues that the choice between `if` and `case` is not stylistic but *semantic*: the two constructs have different execution models, and conflating them produces code that is structurally wrong before it is slow.

1 Introduction

Every POSIX shell programmer knows both `if` and `case`. Most treat them as interchangeable, choosing between them on grounds of readability or personal convention. This paper argues that this view is mistaken, and that the confusion is not merely practical but conceptual.

The POSIX specification defines `if` as executing a *compound command* and branching on its exit status. It does not define a boolean expression evaluator. The expressions that appear inside `if` guards, such as `[ -f file ]` and `[ $x -eq 0 ]`, are not part of the `if` construct. They are *separate commands* whose sole purpose is to encode a truth value as an exit code for `if` to consume.

This encoding is the root of the problem. Every `if [` in a shell script is a *round-trip*: compute a value, serialise it as an exit code, cross a process boundary, and deserialise the exit code as a branch condition. The programmer sees a conditional. The kernel sees a command invocation.

The `case` construct performs no such round-trip. It receives a word, the already-expanded result of any shell expansion, and dispatches on it directly. No external command is run. No exit code is produced or consumed. No process boundary is crossed.

The remainder of this paper establishes this distinction formally in Section 2, develops the theoretical argument in Section 3, and concludes in Section 4.

Notation. Throughout this paper, *command* refers to any executable that may be invoked by the shell, including built-ins. *Process boundary* refers to the `fork/execve` cost incurred when the command is an external binary. *Value dispatch* refers to branching on the content of a variable or arithmetic expression rather than on a command outcome.

2 Two Constructs, Two Execution Models

2.1 if as a Command Evaluator

The POSIX grammar for the `if` command is:

```
if compound-list
then compound-list
[ elif compound-list
  then compound-list ] ...
[ else compound-list ]
fi
```

The guard is a `compound-list`, an arbitrary sequence of commands. The shell executes it and examines the exit status of the last command. Status zero is interpreted as true; any non-zero status is interpreted as false. Nothing in this definition mentions values, types, or boolean expressions. The `if` construct is agnostic to what the guard command *computes*; it cares only about *how it exits*.

The conventional idiom `if [ expr ]` exploits this by using the `[` utility as the guard command. The `[` utility is defined by POSIX as equivalent to `test`: it evaluates a conditional expression and exits with status 0 if the expression is true, and status 1 if false. It exists solely to give `if` something to execute.

This constitutes a layered indirection. The programmer wants to branch on a value. The shell cannot branch on values; it can only branch on exit codes. The programmer therefore invokes `[` to convert the value to an exit code, which the shell then converts back to a branch decision. The value crosses the process boundary twice: once as an argument to `[`, and once as an exit code returned to the shell.

2.2 case as a Native Value Dispatcher

The `case` construct has a fundamentally different grammar:

```
case word in
  pattern [ | pattern ] ... )
    compound-list ;;
  ...
esac
```

The `word` is subject to tilde expansion, parameter expansion, command substitution, and arithmetic expansion, but not word splitting or pathname expansion. The shell matches the expanded word against each pattern in sequence and executes the body of the first match.

No command is run. No exit code is produced or consumed. The dispatch is internal to the shell interpreter. When the word is the result of an arithmetic expansion such as `$(i % 6)`, the shell expands it to a decimal string and matches that string against the literal patterns directly, without parsing or re-evaluating.

This is value dispatch in its native form.

2.3 The Encoding Round-Trip Formalised

The asymmetry between the two constructs can be stated precisely. Given a computed value v and a set of cases C , the execution models are as follows.

if path:

1. Expand $\$(\text{expr})$ to string s
2. Pass s as argument to the `[]` utility
3. `[]` parses s as an integer
4. `[]` evaluates the predicate, exits 0 or 1
5. The shell reads the exit status and selects a branch
6. Repeat from step 1 for each `elif` guard

case path:

1. Expand $\$(\text{expr})$ to string s
2. The shell matches s against the pattern list and selects a branch

Steps 2 through 5 of the `if` path are overhead introduced entirely by the mismatch between what the construct requires (a command exit code) and what the task demands (dispatch on a value). In environments where `[]` is an external binary, steps 2 through 5 additionally require `fork(2)` and `execve(2)`, making each guard evaluation a system call pair. Where `[]` is a shell built-in, the overhead is reduced but not eliminated: argument vector construction, condition evaluation, and exit-code propagation remain. `case` incurs none of these costs on any conformant implementation.

2.4 An Illustrative Example

Consider classifying an integer by divisibility by 2 and 3. The idiomatic `if` version requires up to four $\$(\text{ })$ expansions and four `[]` invocations per iteration:

```
if [  $\$(i \% 2)$  -eq 0 ] && [  $\$(i \% 3)$  -eq 0 ]; then
    result="even and divisible by three"
elif [  $\$(i \% 2)$  -eq 0 ]; then
    result="even"
elif [  $\$(i \% 3)$  -eq 0 ]; then
    result="divisible by three"
else
    result="odd"
fi
```

The `case` version requires exactly one:

```
case  $\$(i \% 6)$  in
    0)    result="even and divisible by three" ;;
    2|4) result="even" ;;
    3)    result="divisible by three" ;;
    *)    result="odd" ;;
esac
```

The single modulo operation suffices because $\text{gcd}(2, 3) = 1$, and the Chinese Remainder Theorem gives:

$$\mathbb{Z}/6\mathbb{Z} \cong \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/3\mathbb{Z}$$

The six residues of $i \bmod 6$ uniquely determine divisibility by both 2 and 3, as shown in Table 1. The **case** version therefore does strictly less work at every level: fewer arithmetic expansions, no `[]` invocations, and no exit-code round-trips. Both reductions follow directly from using the construct that matches the problem’s structure.

Table 1: Mapping of $i \bmod 6$ residues to divisibility class

$i \bmod 6$	$i \bmod 2$	$i \bmod 3$	Classification
0	0	0	even and divisible by 3
1	1	1	odd
2	0	2	even
3	1	0	divisible by 3
4	0	1	even
5	1	2	odd

3 Theoretical Argument

3.1 Exit Codes Are Not Booleans

An exit code is an integer in the range $[0, 255]$ delivered by a process to its parent via `wait(2)`. By Unix convention, zero indicates successful termination and non-zero indicates failure. The **if** construct acts on this convention, treating zero as true and any non-zero value as false.

A boolean is a type with exactly two values. Exit codes are not booleans; they are integers that simulate booleans under a specific convention. This simulation is appropriate when the predicate of interest *is* command success, because success and failure map naturally to the exit-code convention.

When the predicate of interest is not command success but the value of an arithmetic expression, string equality, or any other computed property, the simulation introduces indirection with no semantic justification. The value must be encoded as an exit code by `[]` and immediately decoded by **if**. The round-trip serves no purpose except to satisfy the interface contract of **if**.

This is the precise sense in which **if** `[] ... []` is a category error for value-based dispatch: the tool was designed for command evaluation, and applying it to value evaluation forces an unnecessary encoding step.

3.2 The Correct Use of **if**

The argument above does not condemn **if** in general. The construct is correct and irreplaceable when the condition is genuinely the outcome of a command:

```
if ./configure; then
    make
fi
```

Here the exit code of `./configure` carries information that has no representation as a plain value within the shell. The encoding is not an artifice; it is the natural form of the result. **case** cannot express this. **if** is the correct primitive.

The general principle is: use **if** when the condition is a *command outcome*; use **case** when the condition is a *value*. The two constructs serve different purposes and are not interchangeable.

3.3 Scalability

The overhead of the `if/[` idiom grows linearly with the number of conditional evaluations. A script performing n value-based dispatch operations with `if/[` pays, on every conformant implementation, the cost of argument vector construction, built-in dispatch, and exit-code propagation, multiplied by the number of guards evaluated per dispatch. `case` pays none of these costs.

This scaling behaviour is not implementation-specific. Any POSIX-conformant shell must execute the guard of an `if` statement as a command and inspect its exit status. That requirement is structural; it cannot be optimised away without violating the specification. The overhead is therefore a property of the idiom, not of any particular shell.

4 Conclusion

This paper has argued that the common POSIX shell idiom `if [ condition ]` is a category error: it applies a command evaluator to a value-dispatch problem, incurring an encoding round-trip and, on many implementations, a process boundary that the task does not require. The `case` construct is the semantically correct primitive for value-based dispatch, operating on expanded words directly without invoking any external command.

The title of this paper states the thesis directly: exit codes are not booleans. They are process termination statuses repurposed as truth values by convention. `if` operates on that convention; `case` does not need to. When the condition is a value, `case` is the correct tool.

Ivan Gaydardzhiev, Independent Research, 2026. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).